SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Design Task

Weightage: 20%

QUESTIONS: **Total Marks: 20**

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.
(a) Identify key components required in a navigation bar for an e-commerce website.
(b) Design a responsive navigation bar using appropriate CSS techniques.
(c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).
(d) Evaluate the usability and suggest improvements for better user experience.

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.
(a) Identify suitable CSS properties for hover effects.
(b) Suggest appropriate **CSS animation or transition** for product interaction.
(c) Design a hover effect for a product card.
(d) Evaluate performance and user experience considerations.

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.
(a) Identify layout requirements of a dashboard (rows, columns, alignment).

(b) Compare **Flexbox and Grid** based on layout needs.
(c) Choose the most suitable layout technique and justify your choice.
(d) Evaluate the scalability and responsiveness of the chosen approach.

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.
(a) Identify key sections of a blog layout.
(b) Design a mobile-first layout using CSS.
(c) Demonstrate how the layout scales for larger screens using media queries.
(d) Evaluate accessibility and readability of the design.

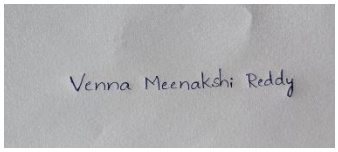
Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.
(a) Identify appropriate JavaScript events for menu interaction.
(b) Design event handling logic for menu toggle functionality.
(c) Implement interaction for user actions (click/hover).
(d) Evaluate performance and suggest improvements.

Code of Conduct:

I **VENNA MEENAKSHI REDDY (192471048)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete	Good understanding	Basic understanding	Poor understanding

		understanding of design requirements and concepts	with minor gaps	with some errors	or incorrect concepts
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Question 1: Responsive Navigation Bar Design

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

(a) Identify key components required in a navigation bar for an e-commerce website.

A responsive e-commerce navigation bar should include:

1. **Company Logo** – Brand identity and home page link.
2. **Navigation Links** – Home, Products, Categories, About, Contact.
3. **Search Bar** – Quick product search.
4. **Shopping Cart Icon** – Displays selected items.
5. **User Account/Login** – Sign in, Register, Profile.
6. **Wishlist** – Save favorite products.
7. **Hamburger Menu** – For mobile devices.
8. **Responsive Layout** – Adapts to desktop, tablet, and mobile screens.

(b) Design a responsive navigation bar using appropriate CSS techniques.

HTML

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="style.css">
</head>
<body>

<nav class="navbar">

<div class="logo">ShopEasy</div>

<ul class="nav-links">
<li><a href="#">Home</a></li>
<li><a href="#">Products</a></li>
<li><a href="#">Categories</a></li>
<li><a href="#">Contact</a></li>
</ul>

<input type="text" placeholder="Search">

<div class="icons">
🛒 👤
</div>

<div class="menu-btn">☰</div>

</nav>
```

```
</body>
</html>
```

CSS

```
*{
margin:0;
padding:0;
box-sizing:border-box;
}

.navbar{
display:flex;
justify-content:space-between;
align-items:center;
background:#333;
padding:15px;
color:white;
}

.nav-links{
display:flex;
list-style:none;
}

.nav-links li{
margin:0 15px;
}

.nav-links a{
color:white;
text-decoration:none;
}

.menu-btn{
display:none;
font-size:28px;
cursor:pointer;
}

@media(max-width:768px){

.nav-links{
display:none;
flex-direction:column;
width:100%;
background:#444;
}
.menu-btn{
display:block;
```

```
}

}
```

(c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).

```
<script>

const menu=document.querySelector(".menu-btn");
const nav=document.querySelector(".nav-links");

menu.onclick=function(){
nav.classList.toggle("active");
}

</script>
.active{
display:flex;
}
```

Working:

- Desktop → Navigation links displayed horizontally.
- Tablet → Items adjust using Flexbox.
- Mobile → Navigation is hidden.
- Clicking the hamburger icon shows/hides the menu.

(d) Evaluate the usability and suggest improvements for better user experience.

Advantages

- Responsive on all devices.
- Easy navigation.
- Search improves product discovery.
- Shopping cart always accessible.

Improvements

- Sticky navigation bar.
- Keyboard accessibility.
- Dropdown product categories.
- Smooth animations.
- High color contrast.
- ARIA labels for screen readers.

Question 2: Product Hover Effect

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

(a) Identify suitable CSS properties for hover effects.

Common hover properties include:

- transform
- transition
- box-shadow
- opacity
- filter
- background-color
- color
- cursor

These properties make products interactive and visually appealing.

(b) Suggest appropriate CSS animation or transition for product interaction.

Use CSS transition for smooth animation.
transition:0.3s ease;
Recommended effects:

- Scale
- Shadow
- Image zoom
- Button color change

(c) Design a hover effect for a product card.

HTML

```
<div class="card">

<img src="product.jpg">

<h3>Wireless Headphones</h3>

<p>$120</p>

<button>Buy Now</button>

</div>
```

CSS

```
.card{
```

```
width:250px;
padding:20px;
border:1px solid #ddd;
text-align:center;
transition:0.3s;
}

.card:hover{
transform:scale(1.05);
box-shadow:0 8px 20px rgba(0,0,0,0.3);
}

button{
padding:10px 20px;
background:blue;
color:white;
border:none;
transition:0.3s;
}

button:hover{
background:darkblue;
}
```

(d) Evaluate performance and user experience considerations.

Advantages

- Smooth transitions improve interaction.
- Shadow gives depth.
- Scaling attracts user attention.

Performance Considerations

- Prefer transform instead of changing width/height.
- Use hardware-accelerated properties.
- Keep animations below **300 ms**.
- Avoid excessive animations on multiple elements.

Question 3: Layout Selection (Flexbox vs Grid)

Suppose that a dashboard layout needs to be designed for a web application.

(a) Identify layout requirements of a dashboard (rows, columns, alignment).

A dashboard typically contains:

- Header
- Sidebar
- Main content
- Cards
- Charts
- Tables
- Footer

Layout requirements:

- Multiple rows and columns.
- Proper alignment.
- Equal spacing.
- Responsive resizing.

(b) Compare Flexbox and Grid based on layout needs.

Comparison

Feature	Flexbox
Layout	One-dimensional
Direction	Row or Column
Alignment	Excellent
Complex Layout	Limited
Dashboard Design	Good

(c) Choose the most suitable layout technique and justify your choice.

CSS Grid is the best choice because:

- Controls both rows and columns.
- Easier dashboard organization.
- Better responsiveness.
- Cleaner code.
- Supports complex layouts.

Example:

```
.dashboard{
display:grid;

grid-template-columns:250px 1fr;
```

```
grid-template-rows:80px auto;

gap:20px;

}
```

(d) Evaluate the scalability and responsiveness of the chosen approach.

Advantages:

- Easy to add widgets.
- Responsive using media queries.
- Supports auto-placement.
- Reduces nested containers.

Example:

```
@media(max-width:768px){

.dashboard{

grid-template-columns:1fr;

}

}
```

Question 4: Mobile-First Blog Layout

Suppose that a blog page must be designed following a mobile-first approach.

(a) Identify key sections of a blog layout.

A blog page should include:

- Header
- Navigation
- Featured Image
- Blog Articles
- Sidebar
- Categories
- Comments
- Footer

(b) Design a mobile-first layout using CSS.

HTML

```
<div class="container">

<header>My Blog</header>

<main>

<article>Blog Content</article>

</main>

<footer>Copyright 2026</footer>

</div>
```

CSS

```
.container{

display:flex;

flex-direction:column;

padding:20px;

}

article{
background:#eee;
padding:20px;

}
```

(c) Demonstrate how the layout scales for larger screens using media queries.

```
@media(min-width:768px){

.container{

display:grid;

grid-template-columns:3fr 1fr;

gap:20px;

}

}
```

```
Desktop:
-------------------------
Header
-------------------------

Article      Sidebar

-------------------------
Footer
-------------------------
```

(d) Evaluate accessibility and readability of the design.

Accessibility:

- Semantic HTML
- Proper heading order
- Alt text for images
- Keyboard navigation

Readability:

- 16–18px font size
- High contrast colors
- Adequate spacing
- Responsive typography
- Short paragraphs

Question 5: JavaScript Event Handling for Dynamic Menu

Suppose that a website includes a dynamic menu that responds to user interactions.

(a) Identify appropriate JavaScript events for menu interaction.

JavaScript Events

Common menu events:

- click
- mouseover
- mouseout
- mouseenter
- mouseleave
- keydown
- touchstart

The **click** event is best for mobile menus.

(b) Design event handling logic for menu toggle functionality.

```
const menu=document.querySelector(".menu");
const nav=document.querySelector(".nav");

menu.addEventListener("click",function(){

nav.classList.toggle("show");

});
```

Logic:

1. User clicks menu icon.
2. JavaScript detects the event.
3. CSS class is toggled.
4. Menu appears or disappears.

(c) Implement interaction for user actions (click/hover).

HTML

```
<button class="menu">☰</button>

<ul class="nav">

<li>Home</li>

<li>Products</li>

<li>Contact</li>

</ul>
```

CSS

```
.nav{

display:none;

}

.show{
```

```
display:block;

}
```

JavaScript

```
const button=document.querySelector(".menu");
const nav=document.querySelector(".nav");

button.onclick=function(){

nav.classList.toggle("show");

}
```

Hover example:

```
.nav li:hover{

background:#ddd;

cursor:pointer;

}
```

(d) Evaluate performance and suggest improvements.

Performance

- Uses a single event listener.
- Toggles CSS classes instead of modifying multiple styles.
- Lightweight and efficient.

Improvements

- Debounce frequent events if needed.
- Add smooth CSS transitions.
- Close menu when clicking outside.
- Support keyboard navigation (Enter, Escape, Tab).
- Use ARIA attributes (aria-expanded, aria-controls) for accessibility.
- Optimize for touch devices.